Data 260
Homework 9

Name: Kshitija Shinde
SJSU ID: 019146491

Value	Definition
SID4	6491
PORT_BASE	8000 + (6491 mod 900) = 8000 + 191 = 8191
PREFIX	"s" + 6491 = s6491
SEED	6491
VERIFY_SEED	260000 + 6491 = 266491
DOMAIN_ID	6491 mod 8 = 3
Assigned Domain	Grocery supply and recall notices

Hardware: MacBook Air, 13-inch, M4, 2025, Apple M4 chip, 16GB RAM, macOS Tahoe 26.5.2
Local model used: qwen3:8b via Ollama
Tagged commit hash: d7135f70502199b15f44d04526d4dfb28b7be534
Repository: https://github.com/Kshitija-Shinde9/data260-6491

Part 1

Scenario

Your system has a primary domain entity (for example a post, product, or patient record). Every time someone accesses one, a counter should go up. Querying the database for every view is slow. Use Redis to make it faster.

Your Task

Add a Redis-backed access/view counter for your primary domain entity. Implement **ONE new method**, for example increment_<entity>_views(<entity>_id), on your database/cache class.

What it should do

1. **Increment the count in Redis on every access.**
   - Use a key that starts with your PREFIX, for example "{PREFIX}:views:<entity>:{id}".
   - Use Redis's INCR command: self.redis_client.incr(key)
2. **Every 10 increments, persist the count to the database.**
   - Check if count % 10 == 0.
   - Update the database row for that entity with the current count.
   - Log a message such as "Synced views to database".
3. **Return the current count.**

Expected Output

```
=== Testing Counter ===

Access #1: Total = 1
Access #2: Total = 2
Access #3: Total = 3
...
Access #10: Total = 10
✓ Synced views to database
Access #11: Total = 11
...
Access #20: Total = 20
✓ Synced views to database
Access #21: Total = 21
...
Access #25: Total = 25
```

Implementation

The method is on RecallCache in code/web_application/recall_cache.py. The prefix is s6491, so recall 1 uses the key s6491:views:recall:1. Opening a notice, GET /api/recalls/{recall_id}, calls the method after the row is loaded. The synced value is stored on recalls.view_count in the MySQL database s6491_rel.

I ran this with scripts/setup_hw09_part1.sh, which starts Redis and then runs scripts/run_hw09_part1.py for 25 accesses of recall 1.

Code

code/web_application/recall_cache.py — the key

```
def view_key(recall_id: int) -> str:
    return f"{PREFIX}:views:recall:{recall_id}"
```

code/web_application/recall_cache.py — increment_recall_views

```
def increment_recall_views(self, recall_id: int) -> int:
    key = view_key(recall_id)
    count = int(self.redis_client.incr(key))
    if count % 10 == 0:
        recall = self.db.get(Recall, recall_id)
        if recall is None:
            raise KeyError(f"no recall with id {recall_id}")
        recall.view_count = count
        self.db.commit()
        log.info("Synced views to database")
    return count
```

code/web_application/routers/recalls.py — called when a recall is opened

```
recall = _get_recall_or_404(db, recall_id)
try:
    RecallCache(db).increment_recall_views(recall_id)
except redis.RedisError:
    pass
return recall
```

code/web_application/models.py — the column that receives the synced count

```
view_count = Column(Integer, nullable=False, default=0)
```

Screenshot

Screenshot 1. Twenty-five accesses of recall 1. The sync line is printed on access 10 and access 20. At the end, Redis key s6491:views:recall:1 is 25 and recalls.view_count in s6491_rel is 20. Accesses 21 through 25 are only in Redis until the next 10th view.



Every access increments Redis. Accesses 10 and 20 also write view_count on the recall row and log “Synced views to database”.

Part 2: Semantic Response Cache

Assignment

Build a semantic cache that sits at the **ingress** of your domain Q&A/agent endpoint, in front of the HW8 agent pipeline.

> **On a cache miss,** the request follows the normal HW8 agent path (local model through the adapter). Store the **query, its embedding, the final answer, and a data-version marker,** then return the answer with is_cached=false.
>
> **On a cache hit,** return the previously stored **final answer** without rerunning the agent pipeline. Return is_cached=true and the **similarity score** alongside it.

Store query embeddings (a model like SentenceTransformers is fine) in Redis with a vector index, and compare queries using **cosine similarity**. Start with a threshold of **0.85**, but do not assume it is optimal. You will examine that in Part 3.

IMPORTANT: Test your system with **at least 10 queries** that include exact duplicates, paraphrased questions, and completely new questions.

Measuring Cache Performance

In your query handler, track whether each response came from the cache:

- When the similarity search finds a match above the threshold, set is_cached=True and log the cache hit with its similarity score before returning.
- When there is no match and you run the agent, set is_cached=False and log the cache miss.
- Add timestamps before and after each operation to measure latency.

Output Format

Return the flag, similarity score, and latency alongside each response. Report:

- Cache hit rate (percentage of queries served from cache)
- Observed average latency for cached vs non-cached queries

Report what you actually measured. **Do not target a predetermined speed-up.**

Implementation

The cache sits in answer_question in code/kafka_service/qa_ingress.py, in front of the HW8 path. A miss calls run_hw8_path: the HW8 planner, then qwen3:8b through src/model_client.py, then the HW8 reviewer. The query, its embedding, the final answer, and a data_version marker are stored in the Redis index s6491_qa_idx. Keys start with s6491:qa:. A hit returns the stored answer and does not call the planner or the model. Similarity is cosine similarity, and the threshold is 0.85.

I ran scripts/setup_hw09_part2.sh. It asked 12 questions: exact repeats, paraphrases, and new questions. The measured cache hit rate was 58.3% (7 hits, 5 misses). Average latency was 221.6 ms for cached answers and 5797.8 ms for non-cached answers. The first blueberry question missed (similarity 0.0000, 14631.7 ms). The exact repeat hit at similarity 1.0000 in 31.3 ms. The strawberry question is not a stored recall. It still hit the blueberry answer at similarity 0.9292, which is above 0.85, so the cache returned that answer.

Code

code/kafka_service/semantic_cache.py - search

```
def search(self, query: str) -> dict | None:
    from redis.commands.search.query import Query

    statement = (
        Query("(*)=>[KNN 1 @embedding $vec AS vector_score]")
        .return_fields("query", "answer", "data_version", "recall_code", "vector_score")
        .sort_by("vector_score")
        .dialect(2)
    )
    try:
        found = self.redis.ft(QA_INDEX).search(
            statement,
            query_params={"vec": embed_text(query).tobytes()},
        )
    except redis.ResponseError:
        return None
    if not found.docs:
        return None
    doc = found.docs[0]
    distance = float(_text(doc.vector_score) or "1")
    similarity = max(0.0, min(1.0, 1.0 - distance))
```

code/kafka_service/qa_ingress.py - hit and miss

```
if found is not None and found["similarity"] >= threshold:
    latency_ms = (time.perf_counter() - started) * 1000
    print(
        f"cache hit similarity={found['similarity']:.4f} "
        f"is_cached=true latency_ms={latency_ms:.1f}",
        flush=True,
    )
    return {
        "answer": found["answer"],
        "is_cached": True,
        "similarity": found["similarity"],
    }

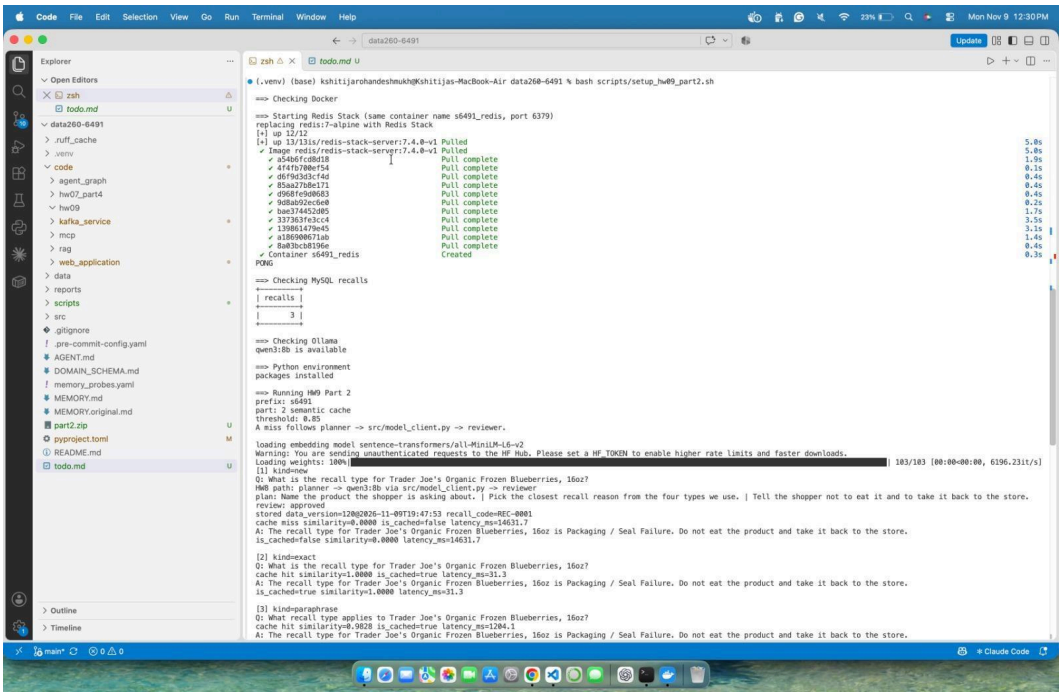
generated = run_hw8_path(question, records)
cache.store(
    question,
    generated["answer"],
    generated["data_version"],
    generated["recall_code"],
)
latency_ms = (time.perf_counter() - started) * 1000
print(
    f"cache miss similarity={best:.4f} is_cached=false latency_ms={latency_ms:.1f}",
    flush=True,
)
```

code/kafka_service/qa_ingress.py - run_hw8_path

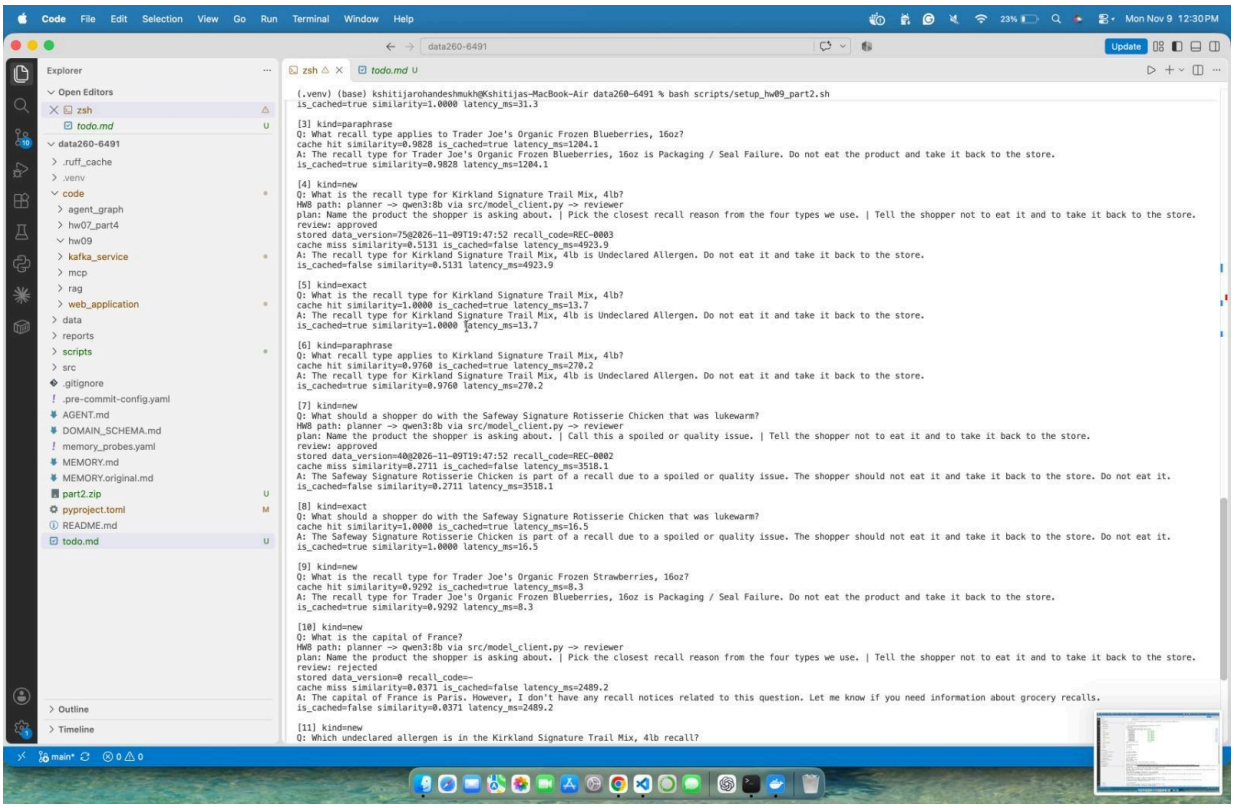
```python
def run_hw8_path(question: str, records: list[dict]) -> dict:
    plan = make_plan(question)
    matched = match_recall(question, records)
    messages = [
        {
            "role": "system",
            "content": (
                "You answer questions about grocery recall notices for s6491. "
                "Use only the records below. If the product is not in the records, "
                "say there is no recall on file for it. Use one or two sentences.\n"
                f"Plan:\n- " + "\n- ".join(plan) + "\n\nRecords:\n" + _fact_sheet(records)
            ),
        },
        {"role": "user", "content": question},
    ]
    client = ModelClient()
    draft = client.complete(messages).content.strip()
    if matched and "do not eat" not in draft.lower():
        draft = draft.rstrip() + " Do not eat it."
    status, answer = review_draft(draft)
```

Screenshots

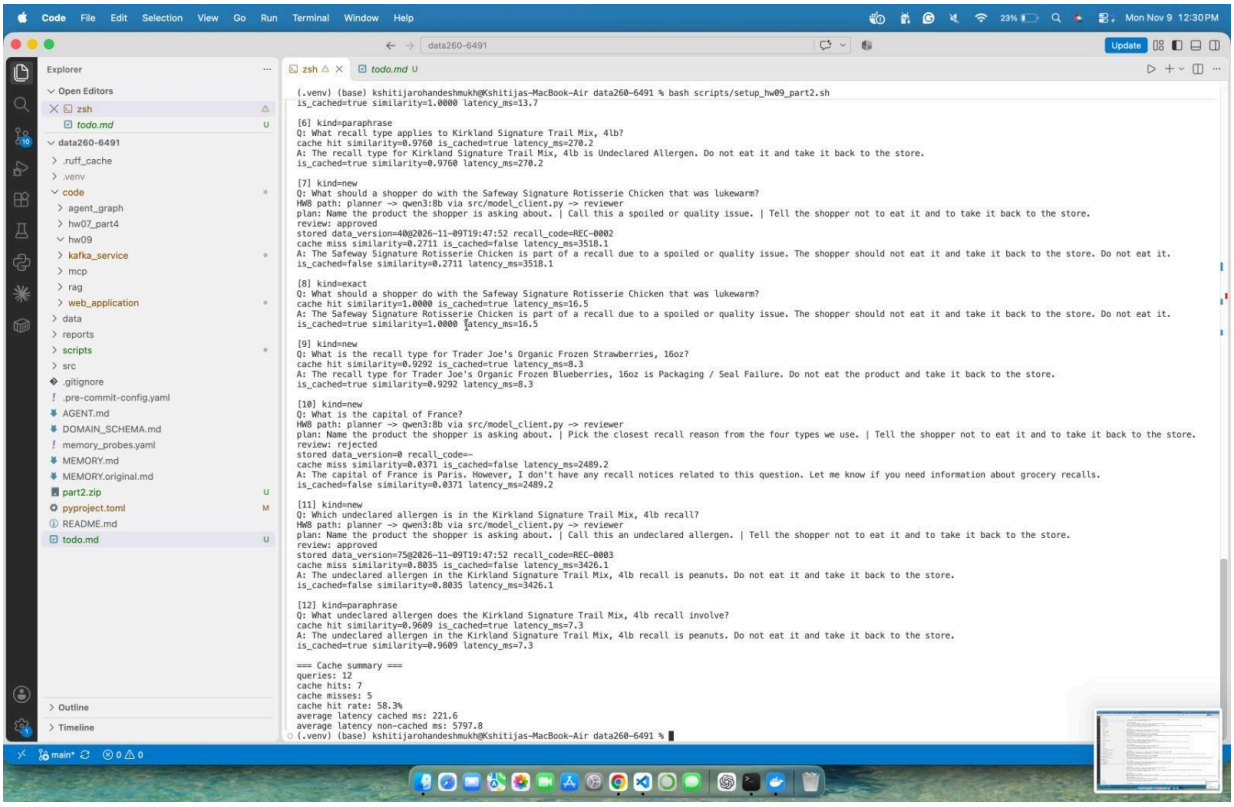
Screenshot 1. Redis Stack is running, the three recalls are in s6491_rel, and qwen3:8b is available. Question 1 is a cache miss and follows planner, then qwen3:8b through src/model_client.py, then the reviewer. Question 2 is the exact repeat: cache hit, similarity 1.0000, is_cached=true.



Screenshot 2. Paraphrases of the blueberry and trail-mix questions hit. The first trail-mix question and the first chicken question miss and store a data_version. Question 9, the strawberry question, hits the blueberry answer at similarity 0.9292.



Screenshot 3. The capital-of-France question misses. The allergen paraphrase hits. The summary is 12 queries, 7 hits, 5 misses, hit rate 58.3%, average cached latency 221.6 ms, average non-cached latency 5797.8 ms.


Code File Edit Selection View Go Run Terminal Window Help
Mon Nov 9 12:30 PM
data260-6491
Update
Explorer
Open Editors
zsh
todo.md
data260-6491
.ruff_cache
.venv
code
agent_graph
hw07_part4
hw09
kafka_service
mcp
rag
web_application
data
reports
scripts
src
.gitignore
.pre-commit-config.yaml
AGENT.md
DOMAIN_SCHEMA.md
memory_probes.yaml
MEMORY.md
MEMORY.original.md
part2.zip
pyproject.toml
README.md
todo.md
Outline
Timeline
zsh
todo.md
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air data260-6491 % bash scripts/setup_hw09_part2.sh
is_cached=true similarity=1.0000 latency_ms=13.7
[6] kind=paraphrase
Q: What recall type applies to Kirkland Signature Trail Mix, 4lb?
cache hit similarity=0.9760 is_cached=true latency_ms=270.2
A: The recall type for Kirkland Signature Trail Mix, 4lb is Undeclared Allergen. Do not eat it and take it back to the store.
is_cached=true similarity=0.9760 latency_ms=270.2
[7] kind=new
Q: What should a shopper do with the Safeway Signature Rotisserie Chicken that was lukewarm?
HW8 path: planner -> qwen3:8b via src/model_client.py -> reviewer
plan: Name the product the shopper is asking about. | Call this a spoiled or quality issue. | Tell the shopper not to eat it and to take it back to the store.
review: approved
stored data_version=40@2026-11-09T19:47:52 recall_code=REC-0002
cache miss similarity=0.2711 is_cached=false latency_ms=3518.1
A: The Safeway Signature Rotisserie Chicken is part of a recall due to a spoiled or quality issue. The shopper should not eat it and take it back to the store. Do not eat it.
is_cached=false similarity=0.2711 latency_ms=3518.1
[8] kind=exact
Q: What should a shopper do with the Safeway Signature Rotisserie Chicken that was lukewarm?
cache hit similarity=1.0000 is_cached=true latency_ms=16.5
A: The Safeway Signature Rotisserie Chicken is part of a recall due to a spoiled or quality issue. The shopper should not eat it and take it back to the store. Do not eat it.
is_cached=true similarity=1.0000 latency_ms=16.5
[9] kind=new
Q: What is the recall type for Trader Joe's Organic Frozen Strawberries, 16oz?
cache hit similarity=0.9292 is_cached=true latency_ms=8.3
A: The recall type for Trader Joe's Organic Frozen Blueberries, 16oz is Packaging / Seal Failure. Do not eat the product and take it back to the store.
is_cached=true similarity=0.9292 latency_ms=8.3
[10] kind=new
Q: What is the capital of France?
HW8 path: planner -> qwen3:8b via src/model_client.py -> reviewer
plan: Name the product the shopper is asking about. | Pick the closest recall reason from the four types we use. | Tell the shopper not to eat it and to take it back to the store.
review: rejected
stored data_version=0 recall_code=—
cache miss similarity=0.0371 is_cached=false latency_ms=2489.2
A: The capital of France is Paris. However, I don't have any recall notices related to this question. Let me know if you need information about grocery recalls.
is_cached=false similarity=0.0371 latency_ms=2489.2
[11] kind=new
Q: Which undeclared allergen is in the Kirkland Signature Trail Mix, 4lb recall?
HW8 path: planner -> qwen3:8b via src/model_client.py -> reviewer
plan: Name the product the shopper is asking about. | Call this an undeclared allergen. | Tell the shopper not to eat it and to take it back to the store.
review: approved
stored data_version=75@2026-11-09T19:47:52 recall_code=REC-0003
cache miss similarity=0.8035 is_cached=false latency_ms=3426.1
A: The undeclared allergen in the Kirkland Signature Trail Mix, 4lb recall is peanuts. Do not eat it and take it back to the store.
is_cached=false similarity=0.8035 latency_ms=3426.1
[12] kind=paraphrase
Q: What undeclared allergen does the Kirkland Signature Trail Mix, 4lb recall involve?
cache hit similarity=0.9609 is_cached=true latency_ms=7.3
A: The undeclared allergen in the Kirkland Signature Trail Mix, 4lb recall is peanuts. Do not eat it and take it back to the store.
is_cached=true similarity=0.9609 latency_ms=7.3
=== Cache summary ===
queries: 12
cache hits: 7
cache misses: 5
cache hit rate: 58.3%
average latency cached ms: 221.6
average latency non-cached ms: 5797.8
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air data260-6491 %
main*
0 0

Part 3: Threshold Economics and Stale Answers

3.1 Labeled test set

Before running any threshold sweep, create and **commit cache_cases.yaml** with **40 labeled domain queries**:

- **10 exact duplicates** (should hit)
- **10 paraphrases** that should hit
- **10 superficially similar** questions with **different correct answers** (should miss)
- **10 unrelated** questions (should miss)

Do not relabel cases after seeing threshold results. If you find a labeling mistake, document it separately instead of editing the file.

3.2 Threshold sweep

Run the 40 cases at thresholds **0.75, 0.80, 0.85, 0.90, and 0.95**. For each threshold, report hit rate, false-hit rate, and mean latency, and fill in the total cost after completing 3.4.

3.3 One false hit in full

Show one false hit completely:

- Both queries (the cached one and the incoming one)
- The similarity score
- The cached answer that was returned
- The correct answer
- A short explanation of the likely embedding confusion (why the model considered the two queries similar)

3.4 Cost model and threshold choice

Create a simple cost model:

- Define a **cache-miss cost** and a **false-hit/support cost**, and justify both.
- Compute the total cost at each threshold.
- Choose one threshold to ship and explain why.

3.5 Stale answers and invalidation

1. Choose one cached query whose answer depends on a specific <PREFIX>_rel record.
2. Update that record so the cached answer becomes stale.
3. Demonstrate the stale hit (the cache returns the old answer).
4. Implement and demonstrate one invalidation strategy using the stored data-version marker or an equivalent design.
5. State the trade-off of your strategy.

Submission

Add screenshots and code for all parts. Include cache_cases.yaml, the filled threshold table, and your written explanations.

Implementation

The 40 labels are in reports/hw09/cache_cases.yaml. I wrote them before the sweep and did not edit the file after seeing the scores. The file has 10 exact duplicates, 10 paraphrases, 10 superficially similar questions, and 10 unrelated questions. I ran scripts/setup_hw09_part3.sh. It seeds the 10 source questions through the HW8 path, then searches the Redis index at each threshold.

A correct hit costs 0.2 because it is one Redis search. A miss costs 8 because the shopper waits for one qwen3:8b call. A false hit costs 40 because returning the wrong recall is worse than waiting for a correct answer. Total cost is correct_hits × 0.2 + misses × 8 + false_hits × 40. Hit rate is cached responses out of 40. False-hit rate is false hits out of the 20 questions that should miss.

Results

Threshold	Hit rate	False-hit rate	Mean latency (ms)	Total cost
0.75	75.0%	50.0%	418.6	484.0
0.80	70.0%	40.0%	456.0	420.0
0.85	62.5%	25.0%	568.9	324.0
0.90	57.5%	15.0%	638.6	260.0
0.95	47.5%	0.0%	836.9	171.8

I ship threshold 0.95. It has the lowest total cost, 171.8, and a false-hit rate of 0.0%. The hit rate falls to 47.5% and the mean latency rises to 836.9 ms, because more questions wait for the model. That is still cheaper under this cost model: a false hit costs 40 and a miss costs 8.

para_08 is labeled as a paraphrase that should hit. It missed only at 0.95. I did not change cache_cases.yaml.

One false hit

Threshold 0.85, case near_01, group superficial.

Cached query: What is the recall type for Trader Joe's Organic Frozen Blueberries, 16oz?

Incoming query: What is the recall type for Trader Joe's Organic Frozen Strawberries, 16oz?

Similarity: 0.9292

Cached answer: The recall type for Trader Joe's Organic Frozen Blueberries, 16oz is Packaging / Seal Failure. Do not eat the product and take it back to the store.

Correct answer: No recall on file for Trader Joe's Organic Frozen Strawberries, 16oz.

Cosine similarity was 0.9292. The questions share these words: 16oz, for, frozen, is, joe's, organic, recall, the, trader, type, what. The incoming question changes: strawberries. all-MiniLM-L6-v2 treats that overlap as almost the same question, but the product is different, so the cached answer is about the wrong recall.

Stale answer

The cached query is “How many units were affected in the Trader Joe's Organic Frozen Blueberries, 16oz recall?” It depends on s6491_rel, recall REC-0001. The first answer says 120 units, and the stored data_version is 120@2026-11-09T20:56:54. I updated affected_units to 300, so the row version became 300@2026-11-09T20:56:56. With the version check off, the cache still returned the 120-unit answer at

similarity 1.0000. With the check on, the run printed invalidated REC-0001 and the fresh answer says 300 units. The script then restored affected_units to 120.

Trade-off: checking data_version on a hit costs one database read, so a hit is no longer only a Redis lookup. The gain is that a shopper does not keep the old unit count after the recall row changes. A time-to-live key would be faster and would stay wrong until it expired.

Code

scripts/run_hw09_part3.py - cost constants

```
MISS_COST = 8.0
FALSE_HIT_COST = 40.0
HIT_COST = 0.2
```

code/kafka_service/qa_ingress.py - invalidation

```
if validate_version and stale:
    cache.delete(found["key"])
    print(
        f"invalidated {found['recall_code']} "
        f"cached data_version={cached_version} current data_version={current_version}",
        flush=True,
    )
    found = None
```

reports/hw09/cache_cases.yaml - one case from each group

```
- id: dup_01
  group: exact_duplicate
  expect: hit
  query: "What is the recall type for Trader Joe's Organic Frozen Blueberries, 16oz?"
  correct_answer: "Packaging / Seal Failure"

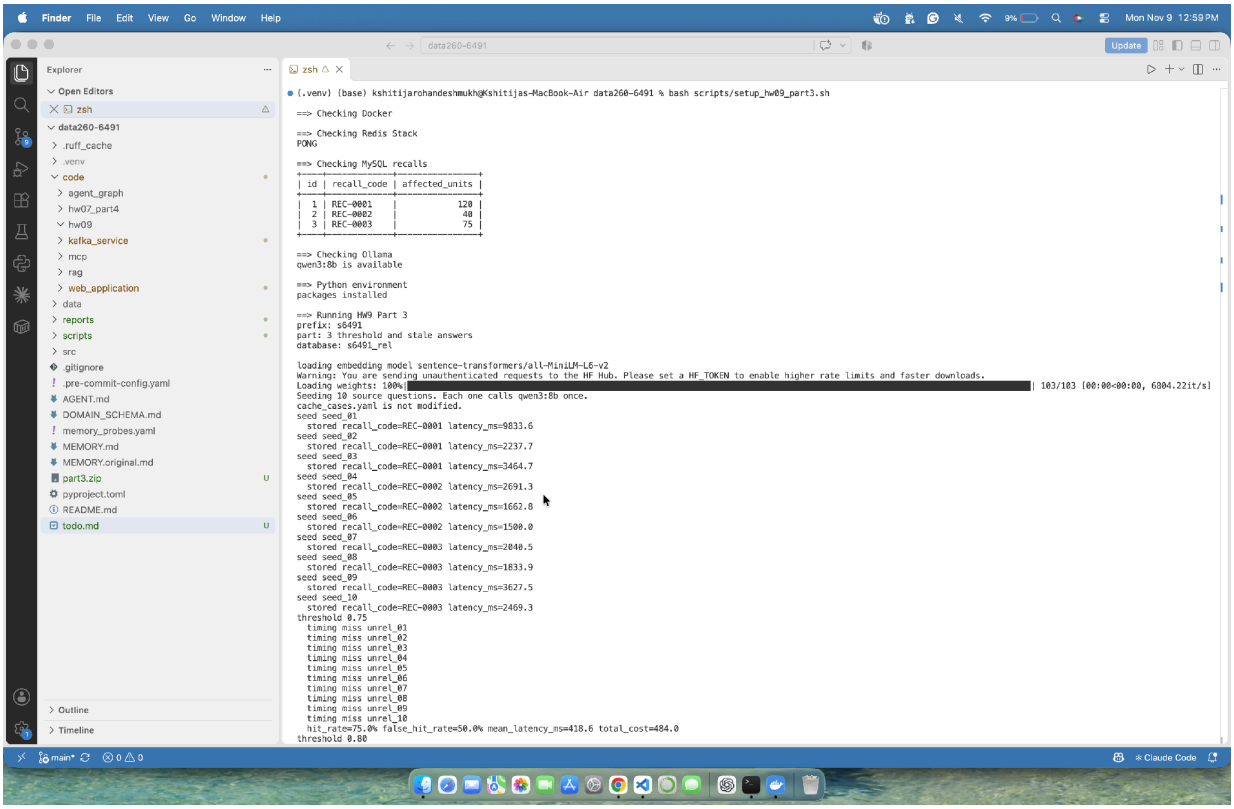
- id: para_01
  group: paraphrase
  expect: hit
  query: "What recall type applies to Trader Joe's Organic Frozen Blueberries, 16oz?"
  correct_answer: "Packaging / Seal Failure"

- id: near_01
  group: superficial
  expect: miss
  query: "What is the recall type for Trader Joe's Organic Frozen Strawberries, 16oz?"
  correct_answer: "No recall on file for Trader Joe's Organic Frozen Strawberries, 16oz."

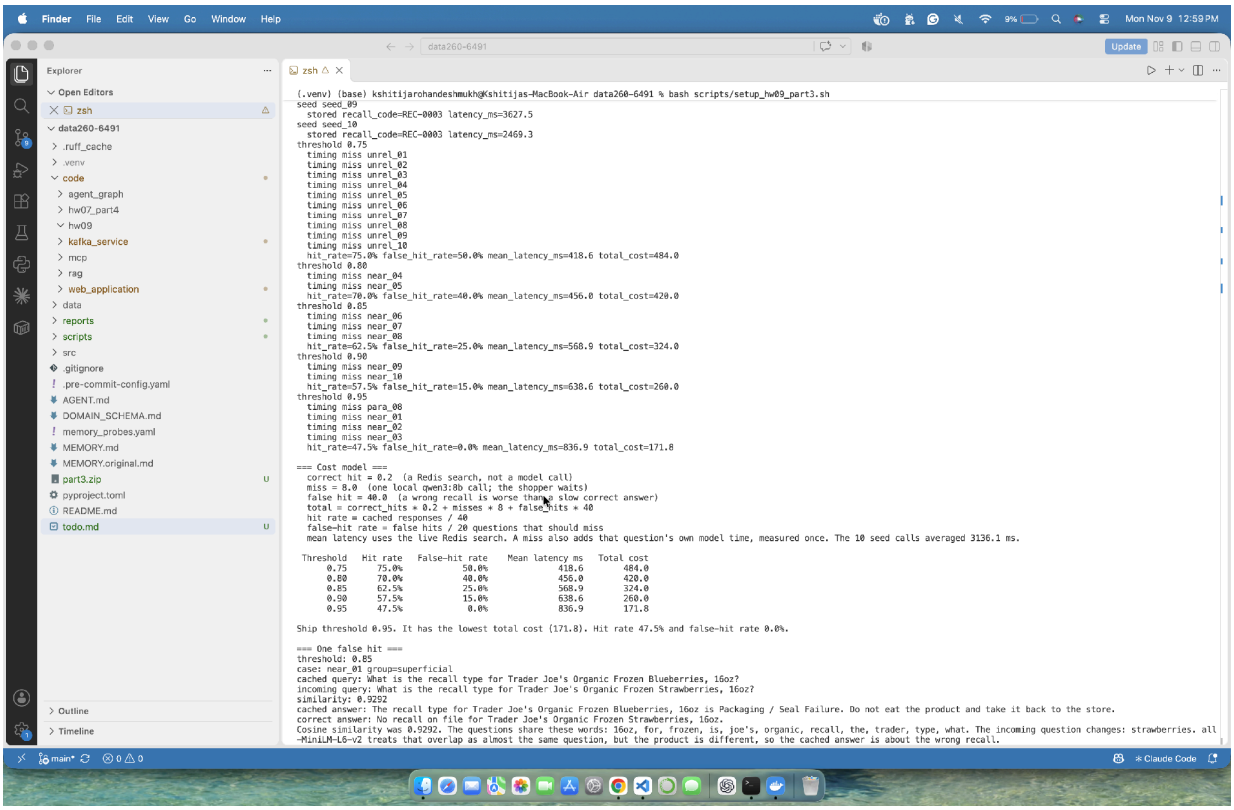
- id: unrel_01
  group: unrelated
  expect: miss
  query: "What is the capital of France?"
  correct_answer: "This is not a grocery recall question."
```

Screenshots

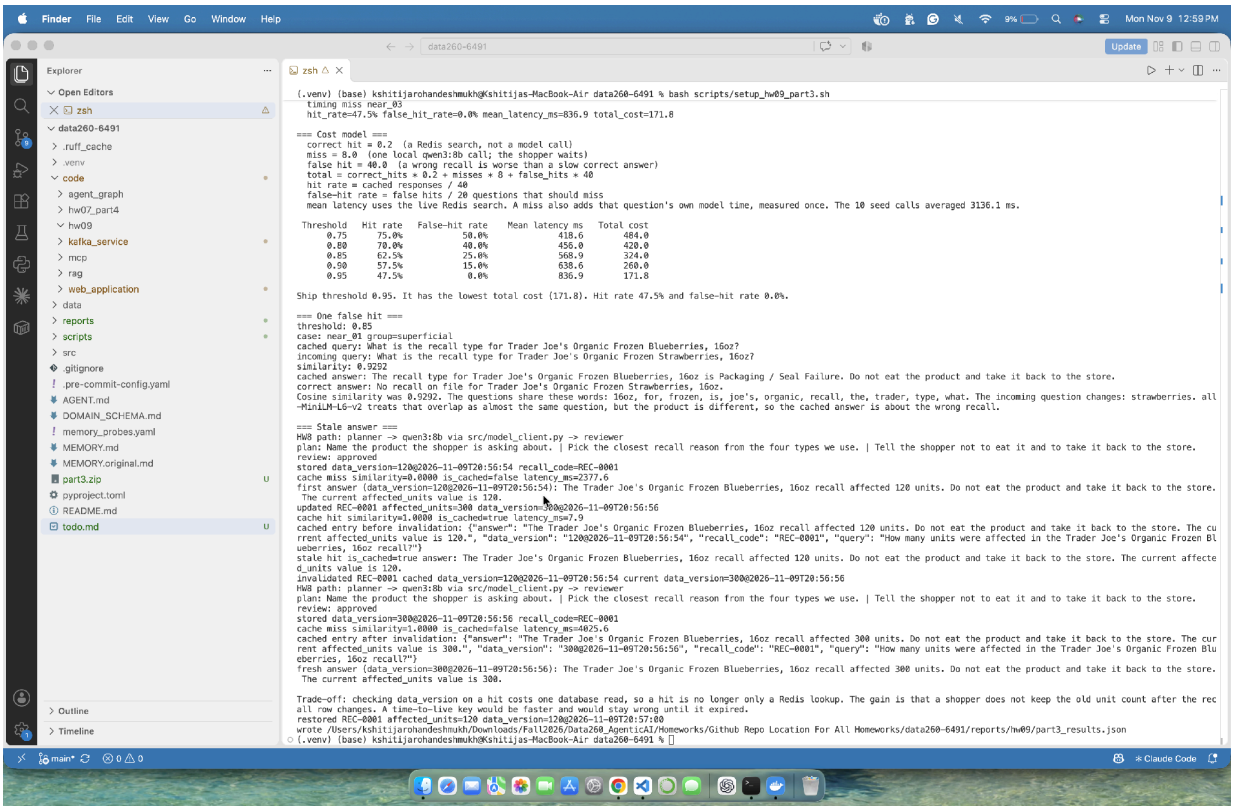
Screenshot 1. Redis answers PONG, s6491_rel has the three recalls, and the 10 seed questions are stored. cache_cases.yaml is not modified. The 0.75 sweep starts.



Screenshot 2. The five thresholds, the cost model, the filled table, the ship line for 0.95, and the start of the false hit.



Screenshot 3. The false hit in full, then the stale answer: 120 units, the hit that still returns 120 after the row becomes 300, the invalidated line, the fresh answer of 300, and the restore to 120.



Part 4: Self-Improving Multi-Agent System

Assignment

Implement a two-agent system consisting of:

1. **Answering Agent** - generates an answer to the user's question.
2. **Evaluator Agent** - evaluates the answer and provides feedback when revision is needed.

The system must support answer revision, reusable lessons, and validation of improvements.

Create a reference dataset in JSON or CSV format based on your assigned domain. The reference dataset must contain the correct domain facts needed to answer the questions in this part. The Evaluator Agent must use this dataset as the ground truth when checking the Answering Agent's responses.

Create the following domain-specific input datasets. These files must contain only the questions or records given to the Answering Agent:

- **training_data.json** - input questions or records used while developing the two agents.
- **validation_data.json** - input questions or records used to determine whether a new lesson improves the system.
- **test_data.json** - held-out input questions or records used for the final evaluation.

For each input, compare the Answering Agent's response with the relevant facts in the reference dataset. Do not include expected answers in the training, validation, or test datasets. Do not use the test dataset to create lessons or modify the system.

4.1 Repair Failed Answers Using Evaluator Feedback

Implement the following workflow:

1. The Answering Agent receives a question and generates an initial answer.
2. The Evaluator Agent checks the answer against the reference dataset.
3. The Evaluator Agent returns:
    - An evaluation result: PASS or REVISE
    - A concise explanation
    - Feedback for correction when the result is REVISE
4. If revision is required, the Answering Agent uses the feedback to generate a revised answer.
5. The Evaluator Agent evaluates the revised answer.
6. The system returns the final answer and evaluation result.

Allow a maximum of two answer attempts. The system must stop after the second attempt and report the final result.

Include at least one controlled test case in which the initial answer is intentionally incomplete or incorrect so that the evaluator-feedback and revision workflow can be demonstrated reliably.

4.2 Save and Reuse Lessons

When the Evaluator Agent identifies a repeated failure pattern, create a reusable lesson.

Store each lesson in lessons.json with:

- lesson_id
- failure_pattern
- correction
- created_at
- use_count

Before generating an answer, the Answering Agent must load relevant lessons and use them when producing the answer.

Example:

```
{
  "lesson_id": "lesson_001",
  "failure_pattern": "The answer omits the current value.",
  "correction": "Check the reference dataset before answering questions about changing values.",
  "created_at": "2026-11-03",
  "use_count": 0
}
```

Demonstrate that a lesson created after one failed answer is loaded and reused for a later related question.

4.3 Validate Improvements Before Keeping Them

Use validation_data.json to evaluate the system before and after adding the lesson.

Keep the lesson only if:

- The failure cases improve or remain unchanged.
- Previously correct answers remain correct.
- Unrelated questions do not receive unsupported answers.

If the lesson causes a regression, document the result and remove or revise the lesson.

Use test_data.json only for the final evaluation after the lesson has been selected.

4.4 Four System Variants

Implement and evaluate the following four variants independently:

1. **Baseline** - Answering Agent without evaluator feedback or lessons.
2. **Evaluator Repair** - Answering Agent revises failed answers using Evaluator Agent feedback.
3. **Lesson Memory** - The system saves and reloads reusable lessons.
4. **Validated Improvement** - A new lesson or rule is kept only after passing the validation dataset.

Run the final test dataset separately on all four variants. Do not change the test dataset between runs.

Report:

Variant, Correct answers, Incorrect answers, Average attempts, Average latency.

Required Demonstration

1. One complete revision sequence showing the question, initial answer, evaluator feedback, revised answer, final evaluation, number of attempts, and final response.
2. One example showing a later question where a previously stored lesson is loaded and reused.
3. The Answering Agent and Evaluator Agent code.
4. The reference dataset, training_data.json, validation_data.json, test_data.json, lessons.json, and the completed results table.
5. A brief explanation of the agent communication, lesson reuse, stopping condition, validation process, and one limitation.

Implementation

The two agents are in code/kafka_service/recall_agents.py. The Answering Agent writes the draft. The Evaluator Agent checks that draft against reports/hw09/reference_dataset.json and returns PASS or REVISE, a short explanation, and feedback when the result is REVISE. The evaluator reads the reference file. It does not call the model.

The reference dataset holds the three recalls from s6491_rel. REC-0001 is Trader Joe's Organic Frozen Blueberries, 16oz, Packaging / Seal Failure, 120 units. REC-0002 is Safeway Signature Rotisserie Chicken, Spoiled or Quality Issue, 40 units. REC-0003 is Kirkland Signature Trail Mix, 4lb, Undeclared Allergen, peanuts, 75 units. training_data.json, validation_data.json, and test_data.json each hold six items, and each item has only an id and a question. scripts/run_hw09_part4.py stops if one of those files contains an answer field.

The first draft of train_repair_01, train_02, train_04, and train_05 is a fixed incomplete sentence, so the revision and the repeated lesson can be shown. Every revision, and every validation and test question, calls qwen3:8b through src/model_client.py. test_data.json is opened only after the lessons have been chosen. I ran this with scripts/setup_hw09_part4.sh.

Revision sequence

Question: What is the recall type for Trader Joe's Organic Frozen Blueberries, 16oz?

Attempt 1 answer: The blueberries were recalled from the store.

Evaluator: REVISE

Explanation: The answer does not state the reference fact: Packaging / Seal Failure.

Feedback: Include the missing fact from the reference dataset: Packaging / Seal Failure.

Attempt 2 answer: The recall type for Trader Joe's Organic Frozen Blueberries, 16oz, is Packaging / Seal Failure.

Final evaluation: PASS. Attempts: 2.

The loop stops after the second attempt. This is the controlled case: the first draft is incomplete on purpose, and the second draft is the revised answer.

Lesson reuse

The recall-type miss on train_repair_01 and again on train_02 created lesson_001. The failure pattern is "The answer omits the recall type." The correction tells the agent to read that product's recall_type and to leave unrelated questions alone.

The later question is train_03: What is the recall type for Kirkland Signature Trail Mix, 4lb?

Loaded lesson_001, use_count=1, created_at=2026-11-09.

Answer: The recall type for Kirkland Signature Trail Mix, 4lb is Undeclared Allergen.

Evaluator: PASS. Attempts: 1.

Validation

validation_data.json was scored before and after the scoped lesson, with repair turned off so the lesson is the only change.

Before the scoped lesson: correct=1, incorrect=5, average attempts 1.0, average latency 1643.5 ms.

After the scoped lesson: correct=3, incorrect=3, average attempts 1.0, average latency 2108.0 ms.

Failures improved, and the previously correct answer stayed correct, so lesson_001 was kept. val_03, "What is the capital of Japan?", stayed a pass both times.

A second lesson was created from the repeated unit-count miss on train_04 and train_05. Its correction is "Always include 120 affected units and the Packaging / Seal Failure recall type in every answer." Because the correction starts with Always, later questions load it.

Before the broad lesson: correct=3, incorrect=3, average attempts 1.0, average latency 2108.0 ms.

After the broad lesson: correct=1, incorrect=5, average attempts 1.0, average latency 3192.7 ms.

REGRESSION: the broad lesson made a previously correct validation answer wrong. val_02 through val_06 were marked REVISE because the answer brought in Packaging / Seal Failure. The broad lesson was removed. It is not written to lessons.json.

Results

The same six questions in test_data.json were scored on all four variants after the lessons were chosen. The file was not changed between runs.

Variant	Correct answers	Incorrect answers	Average attempts	Average latency (ms)
Baseline	1	5	1.00	2741.3
Evaluator Repair	6	0	1.83	6931.5

Lesson Memory	2	4	1.00	4656.3
Validated Improvement	4	2	1.00	3304.5

Evaluator Repair is the only variant that gets a second attempt, so its average attempts are 1.83 and its average latency is 6931.5 ms. It finished 6 correct and 0 incorrect. Lesson Memory reloads the lessons saved before validation, including the broad lesson, so four test answers bring in Packaging / Seal Failure and fail. Validated Improvement keeps only lesson_001. It scores 4 correct and 2 incorrect on the first attempt: the blueberry, chicken, and trail-mix recall types pass, and the Netflix question passes because it is not a recall in the reference dataset. The unit-count and allergen questions still miss, because the kept lesson applies only when the question asks for a recall type.

How the agents work

The Answering Agent writes the draft. The Evaluator Agent then checks that draft against reference_dataset.json and returns PASS or REVISE plus a short reason. On REVISE, the feedback is handed back to the Answering Agent for one revision. The loop stops after two attempts even if the second answer is still wrong.

A lesson is written when the same failure pattern shows up on two training answers. Later questions load that lesson before the draft is written. use_count goes up each time.

Validation runs the same validation_data.json before and after a lesson. A lesson is kept only when previously correct answers stay correct. The broad unit-count lesson is dropped when it pastes the blueberry recall onto other questions. test_data.json is not used to write or edit lessons.

Limitation: the evaluator treats an answer as correct when the reference phrase appears. It does not score fluency. The answering agent also appends a missing required phrase from the record it was allowed to see, so a revision is not only the model's free rewrite.

Code

code/kafka_service/recall_agents.py — Evaluator Agent

```python
def evaluate(self, question: str, answer: str, records: list[dict]) -> dict:
    record = match_record(question, records)
    foreign = _foreign_types(answer, record, records)
    if foreign:
        names = ", ".join(foreign)
        return {
            "verdict": "REVISE",
            "explanation": f"The answer brings in another product's recall type: {names}.",
            "feedback": (
                "The answer adds a recall fact the question did not ask for. "
                "Remove it, and do not name a product that was not asked about."
            ),
            "include": [],
            "pattern": "unsupported",
        }
    if record is None:
        return {
            "verdict": "PASS",
            "explanation": "The question is not about a recall in the reference dataset.",
            "feedback": "",
            "include": [],
            "pattern": None,
        }
    missing = [
        (pattern, phrase)
        for pattern, phrase in _required(question, record)
        if phrase.lower() not in answer.lower()
    ]
    if not missing:
        return {
            "verdict": "PASS",
            "explanation": "The answer includes the reference fact this question asks for.",
            "feedback": "",
            "include": [],
            "pattern": None,
        }
    pattern, phrase = missing[0]
    return {
        "verdict": "REVISE",
        "explanation": f"The answer does not state the reference fact: {phrase}.",
        "feedback": f"Include the missing fact from the reference dataset: {phrase}.",
        "include": [item[1] for item in missing],
        "pattern": pattern,
    }
```

code/kafka_service/recall_agents.py — Answering Agent

```
CONTROLLED_ANSWERS = {
    "train_repair_01": "The blueberries were recalled from the store.",
    "train_02": "The chicken was recalled from the store.",
    "train_04": "Some blueberry packages were affected.",
    "train_05": "Some trail mix packages were affected.",
}

def answer(
    self,
    question: str,
    question_id: str,
    attempt: int,
    lessons: list[dict],
    feedback: dict | None,
    records: list[dict],
) -> tuple[str, float]:
    fixed = CONTROLLED_ANSWERS.get(question_id)
    if fixed and attempt == 1 and not feedback:
        return fixed, 0.0
    record = self._allowed_record(question, lessons, records)
    text, latency_ms = _complete(self._messages(question, lessons, feedback, record))
    if feedback and feedback.get("pattern") == "unsupported":
        return REFUSAL, latency_ms
    if feedback and feedback.get("include"):
        text = _append_missing(text, feedback["include"])
        text = _strip_foreign(text, match_record(question, records), records)
        return text, latency_ms
    if any(is_broad(lesson) for lesson in lessons):
        text = _append_missing(text, ["Packaging / Seal Failure", "120 affected units"])
        return text, latency_ms
    if record is not None:
        text = _append_missing(text, [phrase for _pattern, phrase in _required(question, record)])
        text = _strip_foreign(text, record, records)
        return text, latency_ms
    if asks_fact(question) or _foreign_types(text, None, records):
        return REFUSAL, latency_ms
    return text.strip() or REFUSAL, latency_ms
```

scripts/run_hw09_part4.py — stop after two attempts

```
for attempt in (1, 2):
    attempts = attempt
    answer, spent = agent.answer(
        item["question"], item["id"], attempt, chosen, feedback, records
    )
    latency_ms += spent
    evaluation = evaluator.evaluate(item["question"], answer, records)
    if evaluation["verdict"] == "PASS" or not repair or attempt == 2:
        break
    feedback = evaluation
```

code/kafka_service/lessons.py — load a relevant lesson

```
def select_lessons(question: str, lessons: list[dict], *, increment: bool) -> list[dict]:
    text = question.lower()
    asks_type = "recall type" in text or ("why" in text and "recall" in text)
    chosen = []
    for lesson in lessons:
        if is_broad(lesson) or (asks_type and not is_broad(lesson)):
            chosen.append(lesson)
    if increment and chosen:
        for lesson in chosen:
            lesson["use_count"] = int(lesson.get("use_count", 0)) + 1
    return chosen
```

reports/hw09 — one reference record and one question from each input file

reference_dataset.json

```
{
  "recall_code": "REC-0001",
  "product_name": "Trader Joe's Organic Frozen Blueberries, 16oz",
  "recall_type": "Packaging / Seal Failure",
  "affected_units": 120
}
```

training_data.json

```
{ "id": "train_repair_01",
  "question": "What is the recall type for Trader Joe's Organic Frozen Blueberries, 16oz?" }
```

validation_data.json

```
{ "id": "val_03",
  "question": "What is the capital of Japan?" }
```

test_data.json

```
{ "id": "test_06",
  "question": "How do I reset my Netflix password?" }
```

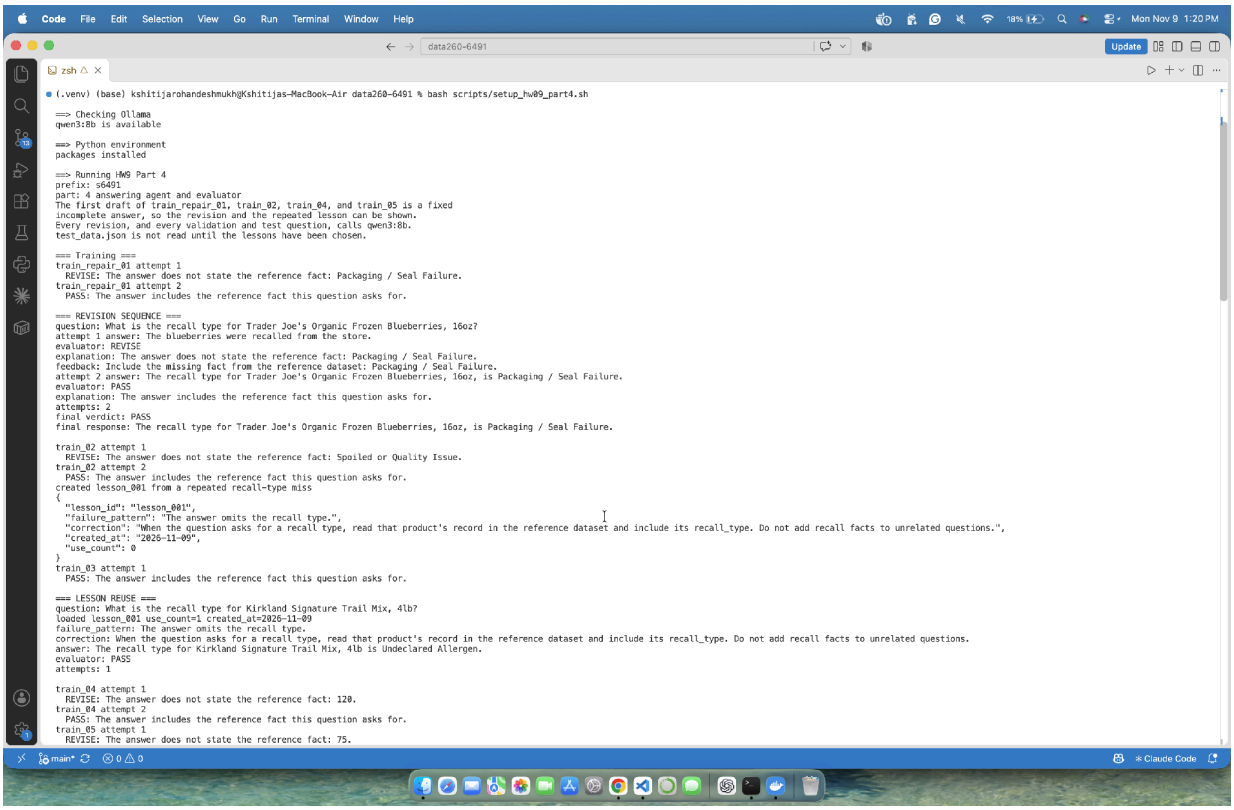
reports/hw09/lessons.json - the lesson that was kept



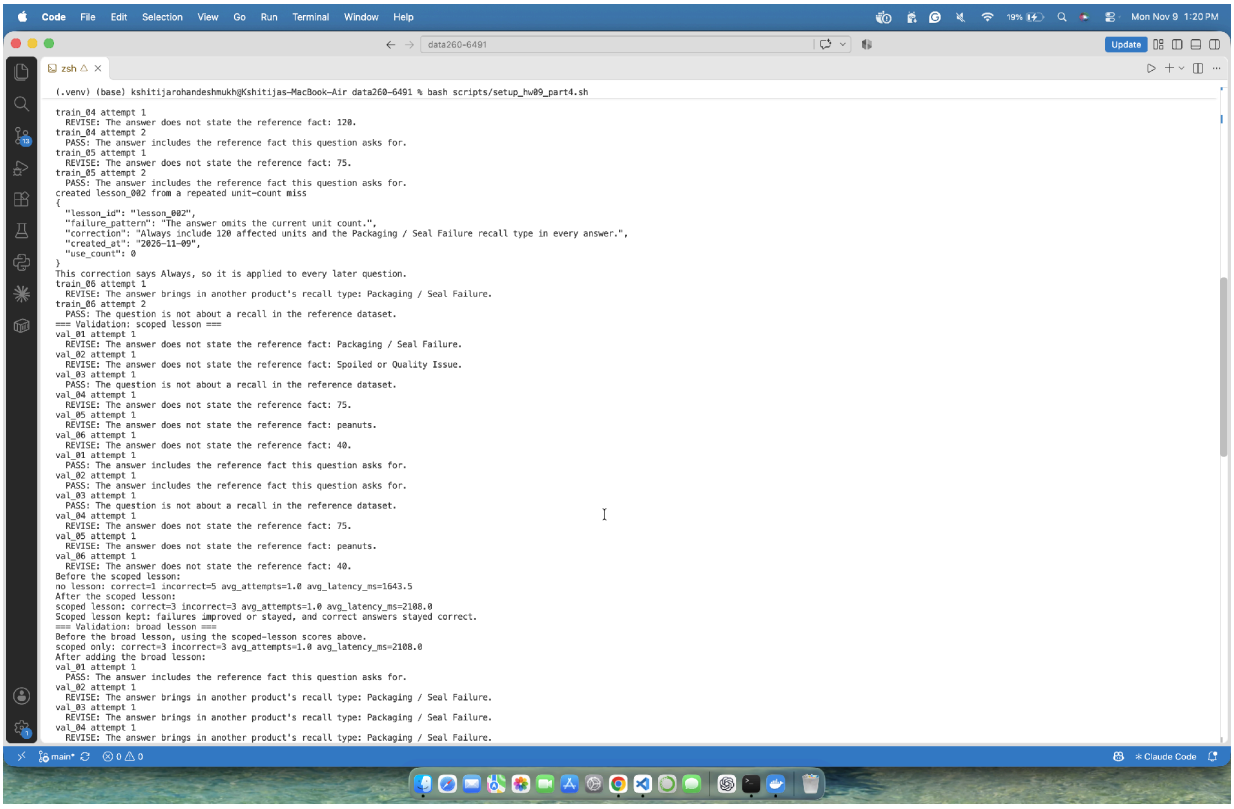
```
{
"lesson_id": "lesson_001",
"failure_pattern": "The answer omits the recall type.",
"correction": "When the question asks for a recall type, read that product's record in the reference dataset and include its recall_type. Do not add recall facts to unrel
"created_at": "2026-11-09",
"use_count": 6
}
```

Screenshots

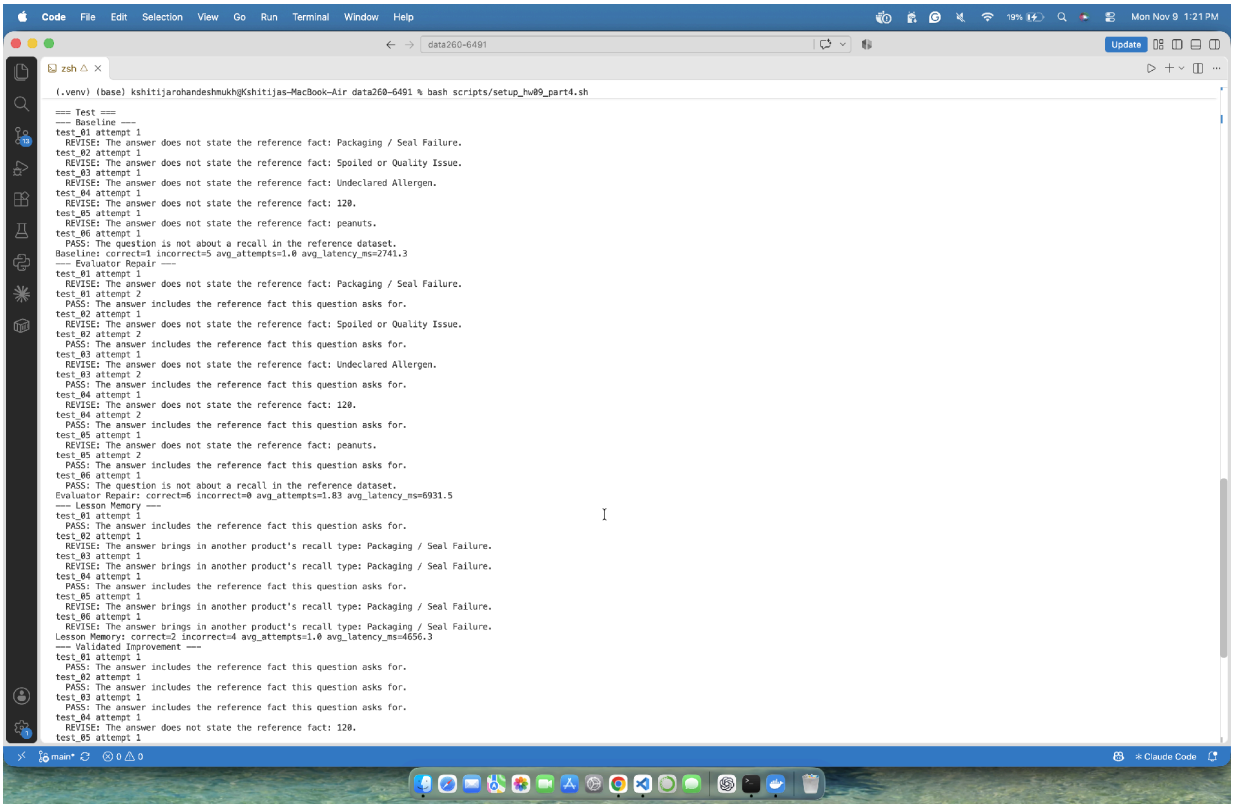
Screenshot 1. qwen3:8b is available. The revision sequence for the blueberry recall type shows the incomplete first draft, REVISE, the feedback, and PASS on attempt 2. lesson_001 is created, and train_03 loads it.



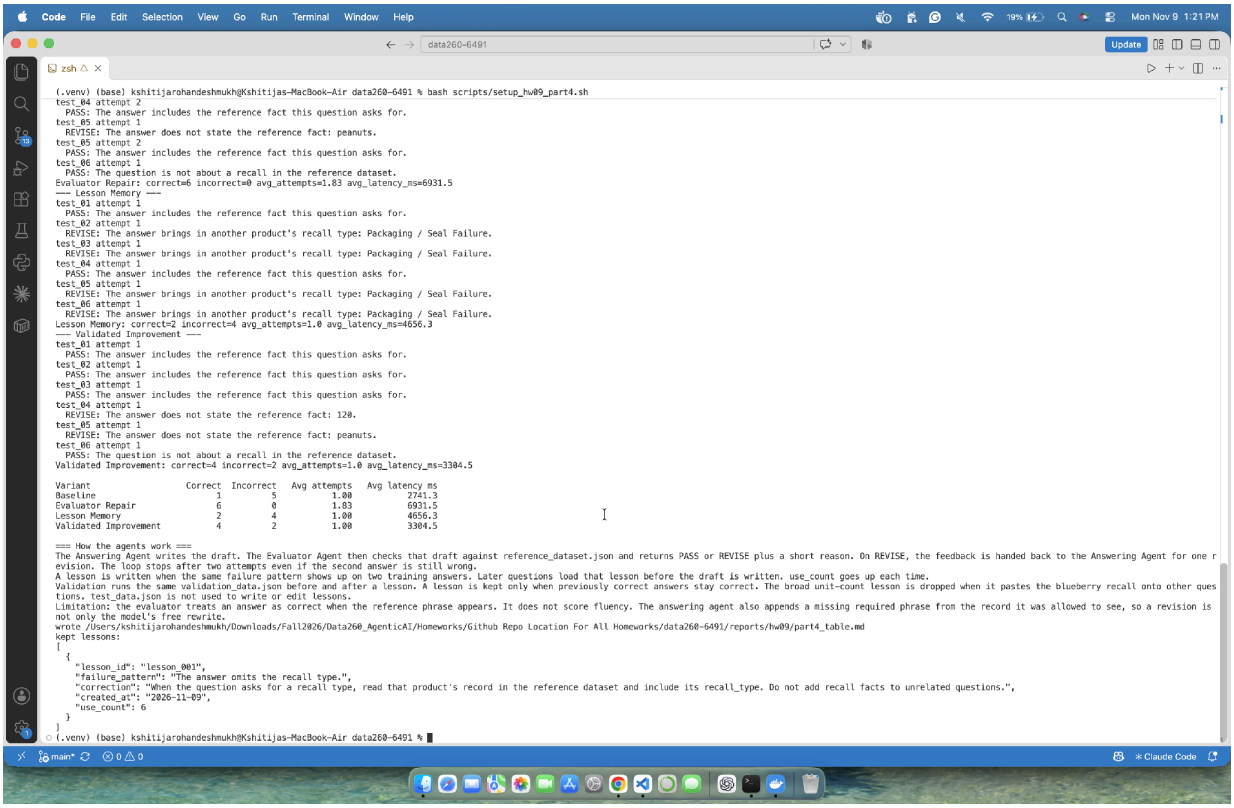
Screenshot 2. train_04 and train_05 miss the unit count, and lesson_002 says Always include 120 affected units and Packaging / Seal Failure. The scoped lesson is kept: correct answers go from 1 to 3. The broad lesson then pastes Packaging / Seal Failure onto the other validation questions.



Screenshot 3. The held-out test set. Baseline is 1 correct and 5 incorrect. Evaluator Repair revises each miss and finishes at 6 correct, 0 incorrect, average attempts 1.83, average latency 6931.5 ms. Lesson Memory starts applying the broad lesson.



Screenshot 4. Lesson Memory is 2 correct and 4 incorrect. Validated Improvement is 4 correct and 2 incorrect, average attempts 1.00, average latency 3304.5 ms. The results table follows, then the kept lesson_001 with use_count 6.


(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air data260-6491 % bash scripts/setup_hw09_part4.sh
test_04 attempt 2
PASS: The answer includes the reference fact this question asks for.
test_05 attempt 1
REVISE: The answer does not state the reference fact: peanuts.
test_05 attempt 2
PASS: The answer includes the reference fact this question asks for.
test_06 attempt 1
PASS: The question is not about a recall in the reference dataset.
Evaluator Repair: correct=6 incorrect=0 avg_attempts=1.83 avg_latency_ms=6931.5
--- Lesson Memory ---
test_01 attempt 1
PASS: The answer includes the reference fact this question asks for.
test_02 attempt 1
REVISE: The answer brings in another product's recall type: Packaging / Seal Failure.
test_03 attempt 1
REVISE: The answer brings in another product's recall type: Packaging / Seal Failure.
test_04 attempt 1
PASS: The answer includes the reference fact this question asks for.
test_05 attempt 1
REVISE: The answer brings in another product's recall type: Packaging / Seal Failure.
test_06 attempt 1
REVISE: The answer brings in another product's recall type: Packaging / Seal Failure.
Lesson Memory: correct=2 incorrect=4 avg_attempts=1.0 avg_latency_ms=4656.3
--- Validated Improvement ---
test_01 attempt 1
PASS: The answer includes the reference fact this question asks for.
test_02 attempt 1
PASS: The answer includes the reference fact this question asks for.
test_03 attempt 1
PASS: The answer includes the reference fact this question asks for.
test_04 attempt 1
REVISE: The answer does not state the reference fact: 120.
test_05 attempt 1
REVISE: The answer does not state the reference fact: peanuts.
test_06 attempt 1
PASS: The question is not about a recall in the reference dataset.
Validated Improvement: correct=4 incorrect=2 avg_attempts=1.0 avg_latency_ms=3304.5
Variant Correct Incorrect Avg attempts Avg latency ms
Baseline 1 5 1.00 2741.3
Evaluator Repair 6 0 1.83 6931.5
Lesson Memory 2 4 1.00 4656.3
Validated Improvement 4 2 1.00 3304.5
=== How the agents work ===
The Answering Agent writes the draft. The Evaluator Agent then checks that draft against reference_dataset.json and returns PASS or REVISE plus a short reason. On REVISE, the feedback is handed back to the Answering Agent for one revision. The loop stops after two attempts even if the second answer is still wrong.
A lesson is written when the same failure pattern shows up on two training answers. Later questions load that lesson before the draft is written. use_count goes up each time.
Validation runs the same validation_data.json before and after a lesson. A lesson is kept only when previously correct answers stay correct. The broad unit-count lesson is dropped when it pastes the blueberry recall onto other questions. test_data.json is not used to write or edit lessons.
Limitation: the evaluator treats an answer as correct when the reference phrase appears. It does not score fluency. The answering agent also appends a missing required phrase from the record it was allowed to see, so a revision is not only the model's free rewrite.
wrote /Users/kshitijarohandeshmukh/Downloads/Fall2026/Data260_AgenticAI/Homeworks/Github Repo Location For All Homeworks/data260-6491/reports/hw09/part4_table.md
kept lessons:
[
{
"lesson_id": "lesson_001",
"failure_pattern": "The answer omits the recall type.",
"correction": "When the question asks for a recall type, read that product's record in the reference dataset and include its recall_type. Do not add recall facts to unrelated questions.",
"created_at": "2026-11-09",
"use_count": 6
}
]
(.venv) (base) kshitijarohandeshmukh@Kshitijas-MacBook-Air data260-6491 %

AI_USE.md - DATA 260 HW9

1. What I used an AI assistant for, and what I did myself

I used AI when I got stuck on the syntax and when I wanted to understand the new concepts like These were the Redis view counter that writes to the database every 10 views, the cosine similarity and the threshold, clearing the cache when a row changes, and the answering agent with an evaluator and a lesson memory. It also helped me find a few errors and talk through the fixes. I wrote all the scripts myself, picked the cost model and the threshold to ship, ran every part on my laptop, took the screenshots, and checked the terminal output against the tables before I put it in the report.

2. AI-produced outputs that were wrong or unsuitable

The first one was in Part 3. When I had to pick a threshold, the AI told me to use 0.85 because it had the best hit rate. It did not think about false hits costing more than a miss.
The second one was in Part 4. My scoped lesson was working, so I asked the AI to make a broader rule. It wrote "Always include 120 affected units and the Packaging / Seal Failure recall type in every answer." But those numbers are only right for one recall, so now the agent gave the blueberry answer to every question.

3. How I detected each problem and verified the fix

For Part 3, I did not just pick the best hit rate. I ran the sweep on all 40 cases and looked at the total cost, where a miss costs 8.0 and a false hit costs 40.0. At 0.85 the false-hit rate was 25% and the cost was 324.0. At 0.95 the false-hit rate was 0.0% and the cost was 171.8, which was the lowest, so I shipped 0.95. The false hit at 0.85 was real and I left it in the report: the strawberry question got the cached blueberry answer at similarity 0.9292.

For Part 4, I tested the broad lesson on validation_data.json before I kept it. The score dropped from 3 correct out of 6 to 1 correct out of 6, which is the same as having no lesson at all. So the broad rule made things worse and I removed it. The scoped lesson had raised the score from 1 to 3 on the same file, so I kept that one. I checked that

lessons.json has lesson_001 and not lesson_002, and I only opened test_data.json after I had made this choice.

4. What I changed, and what the AI wrote in Part 4

In Part 4 the AI suggested the draft answers, the evaluator feedback, and the two lesson corrections. I kept the scoped one, lesson_001, with the failure pattern "The answer omits the recall type." I removed the broad one, lesson_002, after it made the score drop. I did not change the labels in cache_cases.yaml or the questions in the data files after the runs. para_08 is a paraphrase that should hit, and it only missed at 0.95, but I left it as it was instead of changing the label to make 0.95 look better. On the six test questions, Evaluator Repair got all 6 correct but took more attempts and time, and Validated Improvement got 4 correct. Working through the broad lesson that failed is what helped me understand this trade-off.